

MEMORIA DE ANÁLISIS Y DISEÑO: NEXSHOP GROUP S.A.

Módulo de Empresa y Personal

SEDES	EMPLEADOS
id_sede (PK)	Id_empleado (PK)
nombre_sede	dni
ciudad	nombre
tipo_sede	Email_corporativo
	Id_sede (FK - SEDES)

Catálogo y Logística

CATEGORIAS	SUBCATEGORIAS	PRODUCTOS
id_categoria (PK)	id_subcategoria (PK)	id_producto (PK)
nombre_categoria	nombre_subcategoria	referencia
	id_categoria (FK - CATEGORIAS)	nombre_producto
		pvp_actual
		Id_categoria (FK – SUBCATEGORIAS)

STOCKS	TRANSFERENCIAS_STOCK
id_producto (PK) (FK - PRODUCTOS)	id_transferencia (PK)
id_sede (PK) (FK - SEDES)	fecha
cantidad_disponible	cantidad
	id_sede_origen (FK - SEDES)
	id_sede_destino (FK - SEDES)
	id_producto (FK - PRODUCTOS)
	id_empleado_autoriza (FK - EMPLEADOS)

Proveedores

PROVEEDORES	CONDICIONES_PROVEEDOR
id_proveedor (PK)	id_condicion (PK)
nombre_proveedor	precio_coste
id_empleado_representante (FK - EMPLEADOS)	plazo_entrega_dias
	fecha_desde
	fecha_hasta
	id_producto (FK - PRODUCTOS)
	id_proveedor (FK - PROVEEDORES)

Cientes y Fidelización

CLIENTES	DIRECCIONES_CLIENTE	MOVIMIENTOS_PUNTOS
id_cliente (PK)	id_direccion (PK)	id_movimiento (PK)
nombre	calle	cantidad_puntos
apellidos	numero	tipo_movimiento
email	piso	fecha_movimiento
contraseña	codigo_postal	id_cliente (FK - CLIENTES)
fecha_nacimiento	ciudad	id_pedido_online (FK - PEDIDOS_ONLINE)
saldo_puntos_actual	pais	
	id_cliente (FK - CLIENTES)	

Ventas e Incidencias

PEDIDOS_ONLINE	LINEAS_PEDIDO_ONLINE	ENVIOS
id_pedido (PK)	id_linea_pedido (PK)	id_envio (PK)
fecha_pedido	cantidad	numero_seguimiento
estado_pedido	precio_unitario_aplicado	transportista
total_pagado	id_pedido (FK - PEDIDOS_ONLINE)	fecha_estimada_entrega
id_cliente (FK - CLIENTES)	id_producto (FK - PRODUCTOS)	estado_envio
id_direccion_entrega (FK - DIRECCIONES_CLIENTE)		id_pedido (FK - PEDIDOS_ONLINE)
		id_sede_origen (FK - SEDES)

TICKETS_VENTA_FISICA	LINEAS_TICKET_FISICA	DEVOLUCIONES_TIENDA
id_ticket (PK)	id_linea_ticket (PK)	id_devolucion (PK)
fecha_venta	cantidad	fecha_devolucion
total_ticket	precio_unitario_aplicado	motivo
id_sede_tienda (FK - SEDES)	id_ticket (FK - TICKETS_VENTA_FISICA)	id_ticket (FK - TICKETS_VENTA_FISICA)
id_empleado_vendedor (FK - EMPLEADOS)	id_producto (FK - PRODUCTOS)	
id_cliente (FK - CLIENTES) (Acepta NULL)		

TICKETS_INCIDENCIA
id_incidencia (PK)
asunto
descripcion
fecha_apertura
fecha_cierre
estado
nota_resolucion

id_empleado_agente (FK - EMPLEADOS)
id_pedido (FK - PEDIDOS_ONLINE) (Acepta NULL)

Marketing

PROMOCIONES_MARKETING	PRODUCTOS_PROMOCION	VALORACIONES_PRODUCTOS
id_promocion (PK)	id_producto (PK) (FK - PRODUCTOS)	id_valoracion (PK)
nombre_promo	id_promocion (PK) (FK - PROMOCIONES_MARKETING)	puntuacion
descuento_porcentual		comentario
fecha_inicio		fecha_valoracion
fecha_fin		es_verificada
		id_cliente (FK - CLIENTES)
		id_producto (FK - PRODUCTOS)

2. Respuestas a las Preguntas de Reflexión Obligatorias

1. El texto menciona que un pedido online puede generar varios envíos. Explica cómo lo has modelado y por qué. (p. 6)

Respuesta: Lo he modelado mediante una relación de uno a muchos (1:N) entre la tabla Pedidos_Online y una tabla independiente llamada Envios.

- Por qué: Un único registro de pedido puede tener vinculados múltiples registros en la tabla de envíos. Cada registro de envío almacena de forma independiente su propio numero_seguimiento, transportista, fecha_estimada y la id_sede_origen (el almacén o tienda desde donde sale el stock) (p. 3). Esto soluciona de forma limpia el problema de la falta de stock centralizado, permitiendo fragmentar un pedido en diferentes paquetes.

2. Las ventas presenciales y los pedidos online son procesos distintos pero comparten productos y clientes. ¿Has optado por una sola tabla o dos tablas separadas? Justifica tu decisión con ventajas e inconvenientes.

Respuesta: He optado por dos tablas separadas: Pedidos_Online y Tickets_Venta_Fisica.

- Ventajas:
 - Claridad en los datos: Los procesos son operativamente muy diferentes. Los pedidos online necesitan una dirección de entrega, transportista y estados de envío (pendiente, enviado). Las ventas físicas se completan al instante, requieren asociar un empleado cajero/vendedor directo y el cliente suele ser anónimo.

- Evita campos nulos: Separar las tablas evita tener decenas de campos vacíos (NULL) si intentáramos meter ambos procesos en una sola tabla.
- Inconvenientes:
 - Reportes consolidados complejos: Si el departamento de finanzas quiere saber el total de ventas globales de la empresa, tendrá que hacer una consulta utilizando el operador UNION para sumar los datos de ambas tablas.

3. El histórico de precios de un producto y el histórico de promociones son dos cosas distintas. ¿Cómo las diferencias en tu modelo?

Respuesta: Los diferencio separando el precio base del descuento temporal:

1. En la tabla Productos guardo el atributo `pvp_actual`. Para el histórico de precios, propongo una tabla adicional llamada `Historico_Precios_Base` (con campos: `id_producto`, `precio`, `fecha_desde`, `fecha_hasta`) que registra los cambios del valor estándar del producto debido a la inflación o costes.
2. El histórico de promociones se gestiona de forma externa mediante las tablas `Promociones_Marketing` y `Productos_Promocion`. Esta estructura almacena un porcentaje de descuento que se resta temporalmente al precio de venta al público (PVP) durante un rango específico de fechas. Así, el precio final cobrado se calcula cruzando ambas informaciones en el momento de la venta.

4. El saldo de puntos de fidelización "debe poder calcularse siempre desde el histórico de movimientos". ¿Qué implicación tiene esto en el diseño? ¿Guardarías también un campo de `saldo_actual` en la tabla de clientes? ¿Por qué sí o por qué no?

Respuesta: La implicación principal es que es obligatorio diseñar una tabla de auditoría (`Movimientos_Puntos`) donde cada fila registre de manera inmutable cuántos puntos se sumaron o restaron, el cliente, el motivo (pedido o canje) y la fecha.

- ¿Guardaría el campo `saldo_actual` en la tabla `Clientes`? Sí, lo propongo como un campo calculado guardado.
- Por qué: Aunque por estricta teoría de normalización de bases de datos es redundante (ya que el saldo se puede obtener sumando el histórico), a nivel de negocio y rendimiento es una excelente práctica. Si la web tiene miles de usuarios simultáneos consultando su perfil, calcular la suma de todo su histórico en cada clic ralentizaría el sistema. Mantener el `saldo_actual` en la tabla `Clientes` agiliza las consultas web, asegurando mediante restricciones de la aplicación o triggers (disparadores) que este campo se actualice automáticamente con cada nuevo movimiento insertado.

5. Un cliente sin cuenta (compra presencial) puede luego vincularse a una cuenta online. ¿Cómo has previsto esto en tu modelo?

Respuesta: Lo he resuelto permitiendo que el campo `id_cliente` en la tabla `Tickets_Venta_Fisica` acepte valores nulos (NULL).

- Cómo funciona la vinculación: Cuando un cliente compra de forma anónima en una tienda física, se genera el ticket con el campo id_cliente vacío. Si semanas después ese cliente se registra en la web y presenta sus tickets físicos antiguos para reclamar sus puntos, el sistema simplemente realiza una instrucción UPDATE en la base de datos. Se busca el ticket correspondiente y se cambia el valor NULL por el nuevo id_cliente asignado a su cuenta online. Al hacer esto, las compras presenciales se integran inmediatamente a su historial y se disparan los movimientos de puntos acumulados de forma retroactiva.

6. ¿Hay algún requisito del texto que hayas decidido NO implementar o simplificar? Si es así, explica por qué y qué impacto tiene.

Respuesta: He decidido simplificar la gestión logística de las devoluciones online. El texto indica que estas se gestionan como una incidencia y generan un "envío de recogida".

- Simplificación: En lugar de crear una tabla compleja específica para gestionar rutas de camiones de recogida de paquetes, he simplificado el modelo asociando la devolución online directamente a la tabla Tickets_Incidencia a través del estado de la incidencia y añadiendo un registro en la tabla Envios con un tipo especial marcado como "Envío de Recogida".
- Impacto: El impacto es mínimo y positivo. Reduce la complejidad estructural del esquema de la base de datos al reutilizar la tabla de envíos ya existente para los flujos inversos, ahorrando la creación de tablas duplicadas.